

Introduction and motivation

The goal of the project is to explore the Graph Neural Network architectures' capabilities in detecting patterns and outliers in systems engineering models. Systems Engineering (in particular Model Based Systems Engineering) is a fairly young discipline focusing on managing complexity in engineering projects. A typical systems engineering model breaks down a product architecture into System Requirements, System Functions and Logical and Physical Components. Each one of those "domains" is decomposed through tiers representing decreasing levels of abstraction. The critical aspect of Systems Engineering is traceability consisting in the allocation of requirements at different levels of the architecture and ensuring that those requirements are satisfied. A requirement that is missed (either left unallocated or unsatisfied) can in some cases lead to catastrophic accidents leading to the loss of human life. For that reason, robust identification of gaps in traceability can indeed be a matter of life and death.

A classic example of a missed requirement leading to a catastrophic accident is the missed allocation of fatigue load stress testing on the De Havilland Comet fuselage leading to multiple accidents in the 50s and killing dozens of people. (Withey, 2019) The square windows placed in the fuselage caused cracks in their corners due to fatigue following multiple pressurization and depressurization cycles.

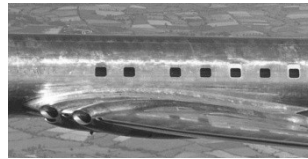


Figure 1 - De Havilland Comet's square windows

Technical background

Given that aerospace data is often proprietary, we had to resort to synthetic data. The first portion of the project consists in the generation of three sets of graphs where the outlier detection grows in complexity:

- Direct satisfaction of allocated requirement – an outlier in this case is any allocate or satisfy relationship that does not coexist with a complementary relationship going the opposite way.



Figure 2 - Simple outliers. Direct satisfaction of allocated requirement

- Decomposition of the allocation target – an outlier in this case is any allocate that does not coexist with a set of decompose and satisfy link whose chain completes the loop or a satisfy link that does not participate in such a loop.

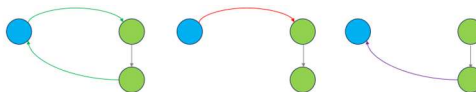


Figure 3 - Standard outliers. Decomposition of the allocation target

- Satisfying the sum of decomposed requirements – an outlier in this case is any allocate relationship where the set of decomposed requirements is not fully satisfied or a satisfy relationship that is redundant.

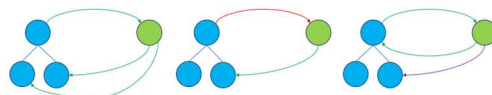


Figure 4 - Complex outliers. Satisfying the sum of decomposed requirements

The synthetic models we generated are sets of graphs where each domain is composed of 50 elements in a hierarchy of 3 tiers for requirements and 4 tiers for functions and components. The plot below shows a simple example with only 6 elements in each domain.

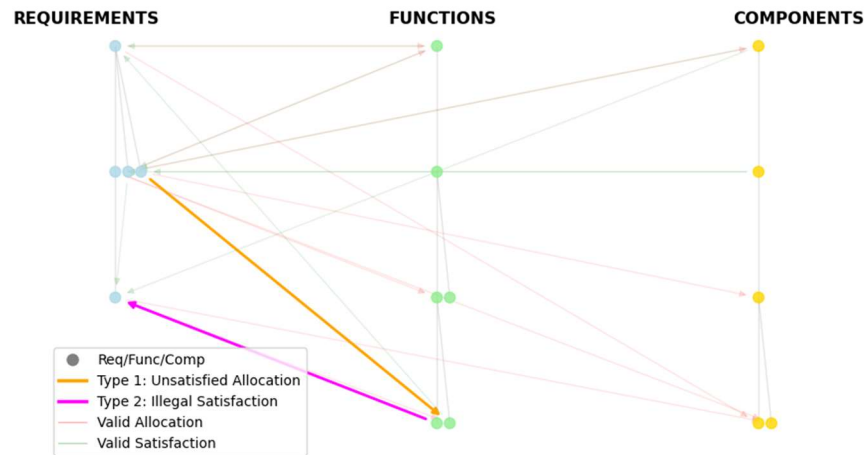


Figure 5 – Simple Systems Engineering graph.

The following plot provides a simplified visualization for the actual size graph where elements are clustered by domain and tier.

Our project will explore the impact of the presence of those outliers on the performance of different GNN architectures and also compare their performance to non-GNN based classifier NN methods. Several papers were published in the recent years on the use of GNNs for detecting and completing missing relationships in Systems Engineering models. One such paper that was particularly well aligned with the idea for our project was recently published by the International Council on Systems Engineering (INCOSE) (E.Karagoz, 2025).

Methodology

GNNs are unique from other machine learning applications in that they act on completely differently structured datasets. The data consists of edges and nodes. An additional challenge is that the same graph can be expressed as many different adjacency matrices depending on node ordering — GNNs are designed to be permutation invariant, meaning the output is consistent regardless of how nodes are arranged.

The architectures evaluated include GCN, GAT, GATv2, GraphSAGE, and RGCN. In this implementation, each architecture uses the same loss function and optimizer. The loss function is binary cross-entropy with logits, appropriate here because each edge is classified independently as either an outlier or valid. To account for the imbalance between valid and outlier edges, a `pos_weight` term scales the loss contribution of outlier edges upward during training. The optimizer is Adam with a learning rate of 0.01.

GCN

All of the GNNs examined in this project share a similar pipeline — and indeed a similar structure to other neural network architectures — those being 1) a multilayer GNN encoder, 2) edge lookup, 3) MLP edge classifier, 4) loss

function, and 5) optimizer. The differences between the architectures are mainly in the first three steps. Figure 7 below illustrates the first three steps for the simplest GNN architecture, the Graph Convolutional Network (GCN).

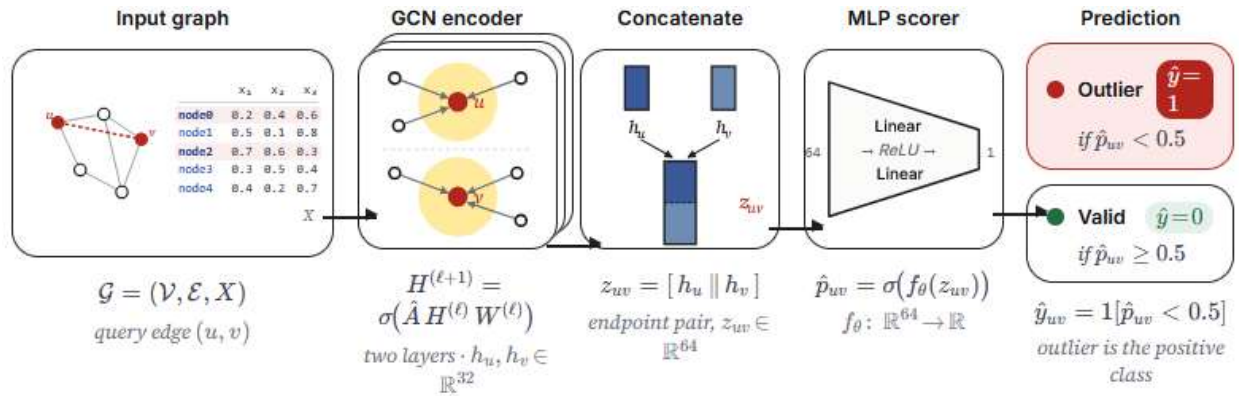


Figure 6 - Edge prediction pipeline of a GCN

An important aspect of the GCN is that the data must be preformatted correctly — this is a key difference from conventional neural networks. The matrix, referred to as X, represents a node on each row. Each column is representative of various features, and these are typically one-hot encoded to describe node type. The order of nodes in X must be maintained for the rest of the pipeline. This order is used to create an adjacency matrix, which describes connections between the different nodes.

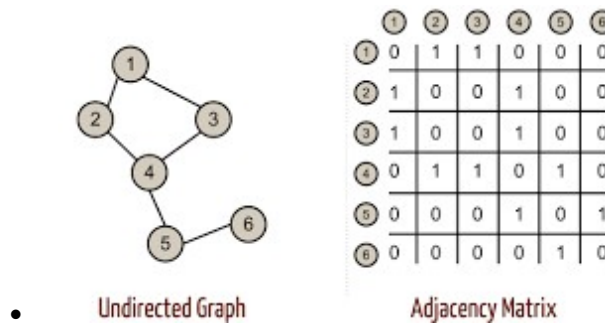


Figure 7 - Adjacency Matrix Example with Simple Undirected Graph

Along with the input matrix X and the adjacency matrix A, the first layer of the encoder uses a trainable weight matrix W, tuned during the training phase. The first layer produces the first embedding values h

$$H = \sigma(\{\hat{A}\}XW) \quad \text{Equation 1}$$

. H is called a node embedding. As seen in the equation above, it accounts for neighboring connections using the adjacency matrix. A key limitation of this is that all edges are treated equally — the adjacency matrix assigns a value of 1 for an edge or 0 otherwise. It is binary and unable to weight edges differently.

The second layer operates the same as the first, but now effectively considers nodes that are two hops away. A two-layer encoder was implemented to start, but more layers could be added to consider further-removed nodes.

After the encoder has produced the final embedding matrix H, it is passed to the MLP edge classifier after being concatenated. Consider two nodes, u and v. The embeddings for u and v are concatenated as $[h_u \parallel h_v]$. H maintains the same row ordering as X, so each row of the matrix corresponds to a node in the same order. At this point in the pipeline, $[h_u \parallel h_v]$ is representative of the edge between nodes u and v. This is passed to the MLP, which consists of two linear layers, one ReLU in between, and one output neuron. This output produces a predicted probability of being an outlier, and is fed into the loss function and subsequently the optimizer. Outside the encoder and MLP, the GNN functions similarly to other neural networks.

GATv2

An important limitation of the GCN, as noted above, is that the edges are all weighted equally. In practice, certain neighbors in a graph will carry more relevant information than others. The Graph Attention Network v2 (GATv2) incorporates this into its approach.

For each directed edge ($i \rightarrow j$), the model computes a raw attention score e_{ij} — a scalar that represents how important neighbor j is when updating node i 's embedding. In GATv2, this score is computed as: $e_{ij} = \mathbf{a}^T \cdot \text{LeakyReLU}(\mathbf{W}[\mathbf{h}_i \parallel \mathbf{h}_j])$, where $\mathbf{W}$ is a learned weight matrix and $\mathbf{a}$ is a learned attention vector. e_{ij} is then normalized across all of node i 's neighbors using softmax to produce α_{ij} — the final attention weight. α_{ij} values for all neighbors of a given node sum to 1, making them interpretable as proportional contributions.

The full pipeline follows these steps: (1) GATv2 Layer 1 — attention scores are computed and neighbors are aggregated, producing node embeddings $\mathbf{H}_1$; (2) GATv2 Layer 2 — a second round of attention-weighted aggregation compresses embeddings to $\mathbf{H}_2$; (3) edge lookup — for each edge ($u \rightarrow v$), the embeddings $\mathbf{h}_u$ and $\mathbf{h}_v$ are concatenated into an edge vector; (4) MLP classifier — the edge vector is passed through two linear layers with a ReLU activation, producing a single logit per edge; (5) prediction — a sigmoid function converts the logit to a probability, thresholded at 0.5 to classify the edge as outlier or valid. The probability is then output to the loss function and optimizer as before. GAT, GraphSAGE, and RGCN were also evaluated; descriptions of each will be provided in the final report.

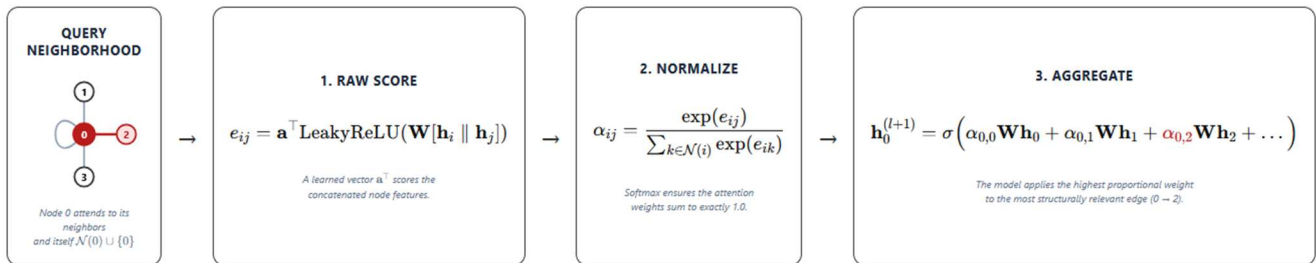


Figure 8 - GATv2 attention mechanism: raw score computation, softmax normalization, and weighted aggregation

The output for each edge is a binary label: 1 = outlier, 0 = valid. Further detail on the edge classification setup will be provided in the final report. Model performance will be evaluated using precision (of the edges flagged as outliers, how many actually are), recall (of all true outliers, how many were caught), and F1-score (the harmonic mean of the two), reported separately for Type 1 and Type 2 outliers.

Preliminary results

The graph data has been successfully converted to PyTorch Geometric format with node features, edge attributes, and binary outlier labels correctly encoded. All five architectures have been implemented and run without errors. The different architecture models are able to run accurate training cycles, and preliminary evaluation confirms each model is learning a meaningful signal rather than predicting a constant class. A preliminary quantitative comparison of architecture performance on unseen graphs can be seen in the figure below.

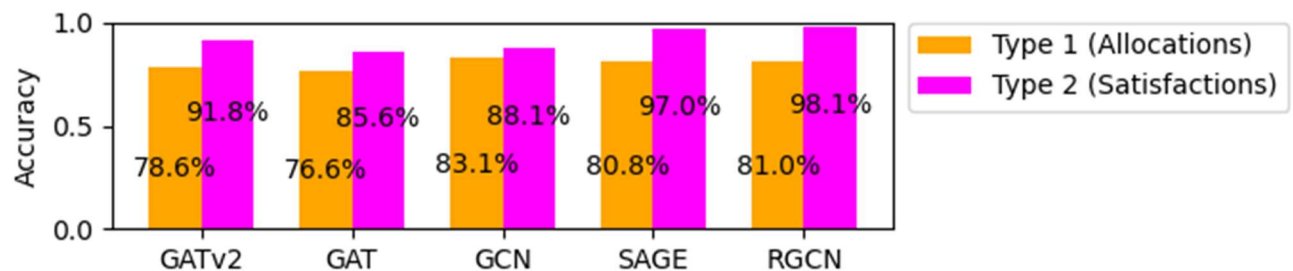


Figure 9 - Accuracy of different GNN architectures on the standard model

Bibliography

E.Karagoz, O. J. (2025). Identification of Missing Knowledge in MBSE System Models Using Graph-Based Machine Learning. *INCOSE*, (pp. 133-149).

Withey, P. (2019, 01 24). *The Real Story of the Comet Disaster*. Retrieved from University of Birmingham:
https://www.fzt.haw-hamburg.de/pers/Scholz/dglr/hh/text_2019_01_24_Comet.pdf

Sanchez-Lengeling, B., Reif, E., Pearce, A., & Wiltchko, A. B. (2021, September 2). A Gentle Introduction to Graph Neural Networks. Retrieved from Distill: <https://distill.pub/2021/gnn-intro/>